

Three root causes produce almost every finding. First, **authorization has no single layer** — permission is checked in a route-level matrix, laboratory scope in a utility module, country scope in a middleware that is never wired to any route, and each controller then improvises on top. Second, **the scope utility fails open**: when it cannot determine a scope it returns an empty filter rather than denying,

contradicting the "deny by default" contract written in its own header comment.

Third, **role is a free-text string** with no enum, no validation, and no single definition — twelve distinct role names appear in the code, eight in the permission matrix, five in the documentation.

Separately and urgently: **production is running code that is not in version control**. The SIS integration API responds live, but its controller, routes and ApiKey model exist only as uncommitted changes on the workstation — 2,574 lines across 47 files, on top of a last commit dated 26 February. Two of the nine critical findings are in that unversioned code.

9 CRITICAL Data crosses lab or role boundaries today	15 HIGH Broken features, absent enforcement, no safety net	8 MEDIUM Operational and durability risk	6 SOUND Verified correct — keep as the pattern
---	---	---	---

01

What is actually deployed

Before the findings, the shape of the system — because several problems are consequences of this shape rather than of any one line of code.

Edge

Apache 2 on 46.19.33.37, vhost injected into httpd-includes.conf to keep DirectAdmin from overwriting it. TLS terminated with the yigini.net combined certificate. HSTS present.

Proxy

ProxyPass / → http://127.0.0.1:3000/, plus a rewrite that upgrades WebSocket requests to ws://127.0.0.1:3000.

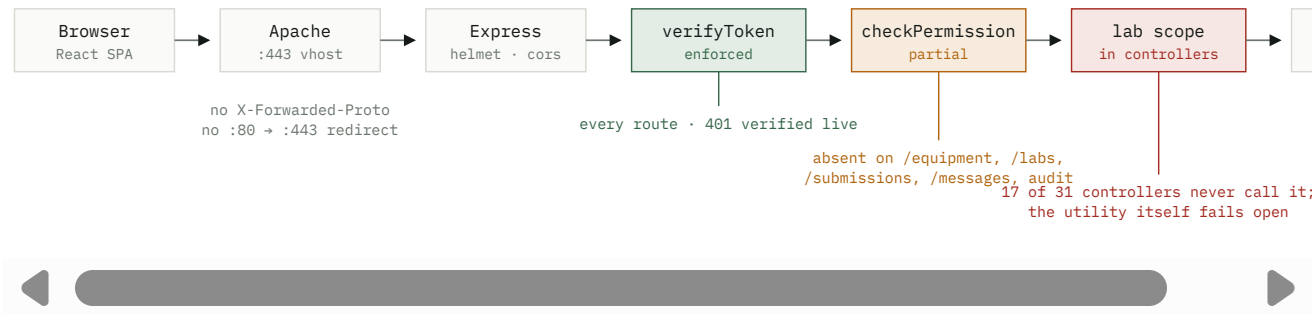
Application

Single Docker container soilfer-lims. Express 5 serves both the JSON API and the built React client from /app/client/dist. Live at audit time, /api/health returning ok, uptime ~15 min.

Database

SQLite via Prisma 7 + better-sqlite3, on a volume that overlays /app/server/prisma. Local copy is 291 MB.

Realtime	ws server on /ws, sharing the HTTP server, authenticated by a JWT in the query string.
Deployment	Manual. scp of hand-picked files, then <code>docker compose build --no-cache</code> , or <code>docker cp</code> of the client bundle straight into the running container. No CI. The only GitHub workflow publishes documentation.
Client build	Deployed <code>index.html</code> last modified today at 15:14 — matches the local build, so the front end is current.



The request path. Authentication is a single well-placed gate; authorization is scattered across three layers and then improvised a fourth time inside each controller.

02 Critical — data crosses a boundary today

Each of these is reachable by an ordinary authenticated user or a valid API key, with no special tooling. Ordered by how easily they are triggered.

ISO-01 CRITICAL

The spectral library is unscoped for every user in every laboratory

server/controllers/spectralController.js:18, 27

`buildScopedWhere()` returns a Prisma clause shaped `{ OR: [...] }`. The caller then tests `baseScope.labId`, a key that clause never has — so the condition is always false and the lab filter is never added to the query.

```
let baseScope = scopeGuard.buildScopedWhere(user, {}, { labField: 'labId' });
if (baseScope.labId) {                                // always undefined
  andConditions.push({ labId: baseScope.labId });    // never runs
}
```

Result: GET /api/spectral returns every MIR/NIR scan held by every laboratory to any logged-in account. The same call is repeated for the scan detail, match-check and batch endpoints in the same controller.

FIX

Merge the returned clause rather than inspecting it — where = scopeGuard.buildScopedWhere(user, where, opts) as the last step, after the other filters are assembled. Then audit every other buildScopedWhere call site for the same misuse.

ISO-02 CRITICAL

The scope guard fails open when it cannot determine a scope

server/utils/scopeGuard.js — buildScopedWhere(), generic branch

The module's header states its philosophy as *"Deny by Default: if no scope can be determined, access is denied."* The implementation does the opposite. For a user with no labId and no country list, the orClauses array stays empty and the function returns {} — an unrestricted query.

Note that the module also exports getLabScope(), which correctly *throws* for a non-SUPER_ADMIN without a lab. buildScopedWhere does not call it; it reads user.labId || null directly and carries on. Roles that are not required to hold a lab — PROJECT_MANAGER, VIEWER, SURVEYOR, MASTER_USER — therefore see all equipment, all projects and all generic entities across the network.

FIX

Make the empty-clause case return an impossible filter ({ id: { in: [] } }) and log it as a misconfiguration, or route the function through getLabScope() so it throws. Never return {} from a security function.

RBAC-01 CRITICAL

A lab manager can read another laboratory's user list by adding a query parameter

server/controllers/userController.js — getUsers()

Scoping is applied first, then user-supplied filters are applied on top of the same object, so

the filter overwrites the security constraint.

```
if (actor.role === 'LAB_MANAGER') { where.labId = actor.labId; } // forced filter
...
if (labId) where.labId = labId; // caller's ?labId= wins
```

GET /api/users?labId=HND-LAB1 from a Guatemala lab manager returns the Honduras roster — usernames, emails, roles, country and project assignments.

FIX

Apply caller filters first, security constraints last, and combine with AND rather than key assignment. This ordering rule belongs in a shared helper, since the same shape recurs elsewhere.

RBAC-02 CRITICAL

Unrestricted mass-assignment on user update, including role and lab

server/controllers/userController.js — updateUser()

The entire request body is passed to Prisma: `const data = { ...updates }`. There is no allowlist. A caller who passes the `canManage()` check can set `labId`, `isActive`, `countries`, `projects` and `mustChangePassword` on the target.

The role guard is narrower still — it only runs `if (actor.role === 'LAB_MANAGER')`. A MASTER_USER can therefore PUT `{ role: 'SUPER_ADMIN' }` onto any user in their country and escalate through them. And because `canManage()` returns true for a lab manager acting on themselves, a lab manager can widen their own `countries` and `projects` arrays at will.

FIX

Explicit field allowlist per actor role. Treat `role`, `labId`, `isActive`, `countries` and `projects` as privileged fields with their own checks, and forbid self-update of any of them.

RBAC-03 CRITICAL

User creation accepts any role string, from any manager role

server/controllers/userController.js — createUser();
server/prisma/schema.prisma:18

The sub-role allowlist `['LAB_TECHNICIAN', 'SAMPLE_RECEPTION', 'VIEWER', 'SURVEYOR']` is applied only when the actor is a LAB_MANAGER. A MASTER_USER — who holds MANAGE_USERS — can create a SUPER_ADMIN outright. There is also no validation that the role string is a real

role: role is String in the schema with no enum and no check constraint, so a typo produces an account that authenticates successfully and is then denied every permission in the matrix.

FIX

A single exported ROLES registry; validate against it on create and update; define, per actor role, which roles that actor may grant. Add a database-level constraint once the value set is settled.

SIS-01 CRITICAL**SIS API keys can read outside their own country and project scope**

server/controllers/sisController.js – buildSisWhere()

The key's restrictions are applied only in the `else` branch of the caller's own filter, so the caller's parameter takes precedence over the key's scope.

```
if (query.country)      where.country = query.country;           // caller wins
else if (sisAuth.countries?.length) where.country = { in: sisAuth.countries };

if (query.labId) where.OR = [{ labId: query.labId }, { assignedLab: query.labId }]
```

A key issued for ["GTM"] returns Honduran samples with `?country=HND`. The `labId` filter is applied with no scope check at all. This endpoint is live in production — `/api/v1/sis/stats` answered during the probe.

FIX

Compute the key's permitted set first, then intersect the caller's filter with it; reject rather than widen when the caller asks for something outside the set.

SIS-02 CRITICAL**Two SIS endpoints ignore key scope entirely**

server/controllers/sisController.js – `getSpectra()`, `getStats()`

`/api/v1/sis/spectra` builds its query as `{ status: 'APPROVED' }` and never consults `req.sisAuth`; `/api/v1/sis/stats` returns global counts the same way. Any valid key — including one scoped to a single country — reads the entire approved spectral library of the whole network.

FIX

Route both through buildSisWhere (once SIS-01 is fixed), joining spectra to their sample for the country and project predicate.

AUTH-01 CRITICAL

Deactivating a user does not deactivate them

```
server/controllers/authController.js – login();
server/middleware/authMiddleware.js – verifyToken()
```

Neither login nor token verification reads `user.isActive`. The login handler checks that the user exists and the bcrypt comparison succeeds, then issues a 24-hour token. `verifyToken` re-fetches the user on every request — so it would cost one line — but only checks existence.

The consequence is that the "toggle staff active" control in Lab Management, and the `isActive` flag generally, have no effect on access. A suspended technician logs in normally. Note that the SIS `apiKeyAuth` middleware *does* check `isActive` — the two auth paths disagree.

FIX

Add `if (!user.isActive) return 401` to both the login handler and `verifyToken`, and to the WebSocket handshake. One line each; do it in the first patch.

OPS-01 CRITICAL

Production is running code that exists only on one workstation

```
git status / git diff against e54b53d (26 Feb 2026)
```

The working tree carries 2,574 changed lines across 47 files that have never been committed — among them the entire SIS integration (controller, routes, API-key middleware, `ApiKey` model), the GloSIS attribute columns on Analysis and Methodology, the impersonation endpoint, and 131 changed lines in `scopeGuard.js` itself. The live probe confirms this code is deployed: `/api/v1/sis/stats` answers with the API-key middleware's error, which does not exist at HEAD.

There is therefore no reviewable record of what is running, no way to diff production against a known state, and no rollback target. Two of the critical findings above (SIS-01, SIS-02) live in code that git has never seen. A workstation failure loses the deployed system.

FIX

Before any remediation: commit the working tree to a branch, push it, and confirm by file

hash that the container's `/app/server` matches that commit. Everything else in this report depends on having a baseline.

03 High — absent enforcement and broken features

These do not leak data on their own, but they are why the system feels unreliable: features that are documented and do not run, roles that cannot do anything, and an authorization design with no place to stand.

RBAC-04 HIGH

The country and project scope middleware is wired to nothing

```
server/middleware/authMiddleware.js — checkScope;  
server/utils/scopeGuard.js — requireLabScope
```

`checkScope` is exported and never imported by any route file. `requireLabScope` likewise has zero call sites. Country and project restriction — the mechanism the user model, the API keys and the admin UI are all built around — is enforced by no middleware at all, only by whatever individual controllers choose to do.

Both would also fail open if they were wired up: each gate is written as `if (target && user.countries.length > 0)`, so an empty array permits everything and an omitted parameter skips the check.

FIX

Delete both, or wire them and invert the conditions. Leaving dead security code in the tree is worse than either — it reads as protection during review.

RBAC-05 HIGH

Seventeen of thirty-one controllers never apply lab scope

```
server/controllers/ — scopeGuard import absent
```

Not imported by: `user`, `submission`, `message`, `notification`, `inventory`, `report`, `dataResults`, `workbench`, `admin`, `analysis`, `kobo`, `sis`, `reviewDecision`, `validation`, `equipmentEligibility`, `auth`, `publicI18n`. Some legitimately need no scope; most handle laboratory-owned records.

FIX

Triage the list into "needs scope" and "global by design", then move the scope decision out of controllers entirely — see the Phase 1 plan.

ISO-03 HIGH

The dashboard reimplements scoping, differently, and fails open

server/app.js — GET /api/dashboard/live

The busiest endpoint in the product hand-rolls its own filter rather than using the scope guard, and scopes samples on assignedLab only, where the guard uses labId OR assignedLab OR country OR assignedTo. Work items are worse:

```
const workWhere = user.labId ? { labId: user.labId } : {}; // no lab ⇒ everything
```

Two definitions of "my lab's data" that disagree also mean dashboard counts will not reconcile with the sample list — a likely source of the "numbers look wrong" class of complaint.

FIX

Replace with the shared guard. One definition of scope, used everywhere, is what makes the counts agree.

RBAC-06 HIGH

Equipment, labs, submissions and messages are authenticated but not authorized

server/routes/equipmentRoutes.js, labRoutes.js:10, submissionRoutes.js, messageRoutes.js, notificationRoutes.js

These routers apply verifyToken and stop. Every equipment endpoint — create, update, change status, log events, approve dispositions, set analysis eligibility — is open to any role, a VIEWER included. GET /api/labs returns the full laboratory register with per-lab user and sample statistics to any account. Audit-log reads (/api/audit-final, /api/admin/audit-direct) are lab-scoped but carry no permission gate, so a technician can read their lab's whole audit trail.

The client compounds this: /equipment is gated on MANAGE_ANALYSES and /admin/audit on MANAGE_USERS, so the UI implies restrictions the API does not have.

FIX

Add the missing `checkPermission` calls, including a real `MANAGE_EQUIPMENT` and `VIEW_AUDIT` key. The general rule: no route reaches a controller without both a permission key and a scope decision.

RBAC-07 HIGH

Two provisioned roles are denied every permission in the matrix

`server/routes/labRoutes.js:268–274;`
`server/middleware/authMiddleware.js – PERMISSIONS`

Creating a laboratory provisions six accounts, two of which are `AUDIT_USER` and `EXTERNAL_VIEWER`. Neither string appears anywhere in the permission matrix, so `checkPermission` denies them every action — including `VIEW_SAMPLES`. These users authenticate successfully and then get 403 on essentially every screen.

The wider problem is that role has no single definition. Twelve strings appear in the code (`SUPER_ADMIN`, `MASTER_USER`, `LAB_MANAGER`, `LAB_TECHNICIAN`, `SAMPLE_RECEPTION`, `PROJECT_MANAGER`, `SURVEYOR`, `VIEWER`, `AUDIT_USER`, `EXTERNAL_VIEWER`, `COUNTRY_ADMIN`, `ADMIN`); eight are in the matrix; five are in the administrator guide. `ADMIN` survives as a legacy alias in four `projectController` branches and one in `sampleController` that nothing can ever satisfy, and `COUNTRY_ADMIN` has a whole permission branch in `userController` that is unreachable because the role is not in the matrix.

FIX

One `roles.js` registry: the canonical list, each role's permission set, and a display name. Import it in the middleware, the client, the seed and the lab-provisioning routine. Migrate or delete the orphan strings.

INT-01 HIGH

KoboToolbox auto-sync does not exist

`server/services/koboService.js;`
`server/controllers/koboController.js:211; server/app.js:91`

The README advertises "automatic synchronization with KoboToolbox". The configuration model stores a `syncIntervalMins` per laboratory, defaulting to 15, and `syncAll` is annotated "*used by scheduler*". There is no scheduler. The only `setInterval` in the server is the daily database backup. Field submissions arrive only when someone manually calls `POST /api/kobo/sync-all` — and that route requires `MANAGE_ANALYSES`, so reception staff cannot trigger it either.

FIX

A single scheduler tick that reads active configs and syncs each lab on its own interval, with a per-lab last-run and last-error record surfaced in the admin UI, so a failing connection is visible rather than silent.

INT-02 HIGH

The Kobo client has no timeout, pagination or retry

`server/services/koboService.js` – `fetchSubmissions()`

A bare `axios.get` with no `timeout` option: if the Kobo server stalls, the Express handler waits indefinitely and holds a request slot. `data.json` is fetched without `limit` or `start`, so large forms are truncated at the server's page size with no indication that results are incomplete. A transient failure aborts the whole batch with no retry and no partial-progress record.

FIX

Explicit timeout (30 s), paginate until exhausted, exponential-backoff retry on 5xx and network errors, and persist the highest submission id processed so a resumed sync does not refetch from zero.

INT-03 HIGH

Public report PDF download is broken — Puppeteer is not installed

`server/controllers/reportController.js` – `getPublicReportPdf()`;
`server/package.json`

The handler does `require('puppeteer')` inside a try/catch. `puppeteer` is not in dependencies and is not in `node_modules`, so the catch always fires and the endpoint returns `{ format: 'pdf-data', content, version }` — JSON, with HTTP 200 and a console warning nobody reads. The client has no handler for that shape: no occurrence of `pdf-data` exists anywhere in `client/src`. A recipient clicking "Download PDF" on a shared report link gets nothing useful.

FIX

Decide the strategy rather than keeping the fallback. Server-side is better for a customer-facing document: add `puppeteer-core` plus Alpine's `chromium` package in the Dockerfile and point `executablePath` at it. Then make the missing-browser branch a 503, not a 200 — a silent success is why this went unnoticed.

WS-01 HIGH**Every realtime event is broadcast to every connected user in every laboratory**

server/wsServer.js – broadcastToAll; 20+ call sites in spectralController, workItemController, workbenchController

broadcastToUser and broadcastToUsers exist and are correct, but the work-item and spectral paths call broadcastToAll. Every WORKITEM_UPDATE, WORKITEM_CHANGED and SPECTRAL_UPDATE — carrying sample id, work-item id and status — goes to every socket on the server.

Two consequences. It leaks activity metadata across laboratories. And because the client hooks refetch on these events, a busy laboratory triggers continuous refetches in every other laboratory's browser — which will present to users as unexplained flicker, lost scroll position and general sluggishness.

FIX

Track labId on the socket at handshake and add broadcastToLab(labId, ...). Replace every broadcastToAll call in those three controllers.

WS-02 HIGH**WebSocket handshake carries the token in the URL and skips the user check**

server/wsServer.js – init()

The client connects to /ws?token=<jwt>. Apache's CustomLog ... combined records the full request line, so every access-log entry contains a valid 24-hour bearer token. The handshake verifies the signature but never loads the user, so it accepts tokens for deleted or deactivated accounts. The CONNECTED frame then returns onlineUsers: Array.from(clients.keys()) — the ids of every user online across the network.

FIX

Move the token to the Sec-WebSocket-Protocol header, load and validate the user in the handshake, and filter the online list to the caller's laboratory.

AUTH-02 HIGH**Forced password change is enforced only in the browser**

client/src/components/ForcePasswordChangeModal.jsx;
server/middleware/authMiddleware.js

mustChangePassword is set on every created user and on every administrator-set password, but no server middleware consults it. The token issued at login is fully privileged; anything that talks to the API directly bypasses the modal. Newly provisioned lab accounts ship with a shared default password, which makes this reachable in practice.

FIX

In `verifyToken`, reject with 403 and a `PASSWORD_CHANGE_REQUIRED` code for every route except `/api/auth/me` and `/api/auth/change-password`.

AUTH-03 HIGH**Impersonation is untraceable**

`server/controllers/authController.js` – `impersonate()`

The endpoint mints a token for the target user that is byte-for-byte indistinguishable from a genuine login: same claims, no actor claim, no shortened lifetime. Nothing is written to the audit log. Every action taken while impersonating is attributed to the target user with no way to establish that an administrator was driving. For a laboratory system that keeps a chain of custody and signs result reports, that is a records-integrity problem as much as a security one.

The endpoint is also open to `MASTER_USER`, despite its own error message reading "Only Super Admins can impersonate users", and it permits impersonating another `SUPER_ADMIN`.

FIX

Add an `act` claim naming the real actor; propagate it into `auditLogger` so every entry records both identities; cap the lifetime at 30 minutes; write an `IMPERSONATE_START` audit entry; block impersonation of equal-or-higher roles; show a persistent banner in the client.

AUTH-04 HIGH**Tokens cannot be revoked**

`server/controllers/authController.js`; `server/config/auth.js`

Tokens carry no `jti` and no version, and there is no denylist. Logout clears `localStorage` and nothing else. A leaked token — from an access log, a shared machine, a browser extension — is valid for 24 hours with no way to cut it off short of rotating `JWT_SECRET`, which signs out every user on the system.

FIX

Add a `tokenVersion` integer to `User`, include it in the token, compare it in `verifyToken`, and

increment it on password change, deactivation and forced logout. Cheap, and it turns AUTH-01 into a complete control.

QUAL-01 HIGH

The whole test suite has never been able to run

```
server/tests/**/*.test.js - require('../setup') /
require('../../setup')
```

All eleven test files import `getAuthToken` or `generateToken` from `tests/setup.js`. That file does not exist. Every suite fails at import, including

`tests/security/rbac_enforcement.test.js` — which was written specifically to catch this class of bug, and whose comments already name the vulnerabilities ("*1. ADMIN ROUTES (Currently Vulnerable)*") that this audit independently confirms.

There is also no CI. The only GitHub Actions workflow builds the documentation book.

FIX

Write `tests/setup.js` against a seeded temporary SQLite file, get the eleven existing suites green, then add one regression test per critical finding in this report before fixing it. This is the cheapest single item here and it protects everything else.

OPS-02 HIGH

Deployment is a manual file copy with no gate and no rollback

```
.agent/workflows/deploy.md
```

The documented procedure is `scp` of individually named files to `/opt/lims/server/`, then a container rebuild — or, for the front end, `docker cp` of `dist` directly into the running container, with a manual `rm -rf` first because `docker cp` is additive and stale assets otherwise accumulate. The schema must be uploaded to two separate paths because a volume overlays the Prisma directory, and the entrypoint is overridden so its schema push and seed never run.

Every one of these steps is a place where production and the repository can diverge silently, and OPS-01 shows that they have. There is no build gate, no test gate, no image tag, and no previous version to roll back to.

FIX

Build one tagged image in CI from a commit; ship that image; keep the previous tag for rollback. The volume-overlay workaround disappears once the schema lives in the image and migrations run as an explicit step.

04 Medium — durability and operational risk

DATA-01 MEDIUM

Schema changes are pushed, not migrated

```
server/prisma/migrations/ – one entry, 20260122112435_init;  
.agent/workflows/deploy.md
```

A single migration from 22 January exists. Every schema change since — including the ApiKey model and the GloSIS columns now in production — was applied with `prisma db push`, which leaves no record of what changed or when. `schema_proposed.prisma` sits beside the real schema with no indication of which is authoritative. There is no way to reproduce the production schema from the repository, and no way to reverse a change.

FIX

Baseline the current production schema as a migration, then use `prisma migrate deploy` as an explicit, logged deployment step. Delete `schema_proposed.prisma`.

DATA-02 MEDIUM

A 291 MB SQLite file with a file-copy backup is at its limits

```
server/prisma/dev.db; server/scripts/backup_db.js; server/app.js:88-96
```

SQLite admits one writer at a time. With spectral payloads stored inline, a multi-laboratory deployment, and WebSocket-driven refetch storms (WS-01) amplifying read load, write contention is a plausible cause of intermittent slowness and timeouts. The backup is a `setInterval` in the application process that copies the file every 24 hours — it does not survive a restart cycle in any predictable phase, has no WAL checkpoint, no integrity verification, and no off-host copy. The container restarted roughly fifteen minutes before this audit; each restart resets that timer.

FIX

Short term: replace with `VACUUM INTO` on a cron outside the app process, verify each backup opens, and copy off-host. Medium term: evaluate PostgreSQL — Prisma makes the swap mostly mechanical, and it removes the write-contention ceiling along with the volume-overlay workaround.

NET-01 MEDIUM**The Apache vhost drops the scheme and never redirects to HTTPS**

```
deploy/setup_da_proxy.sh; deploy/httpd-lims.conf
```

Neither vhost sets X-Forwarded-Proto, so with `trust proxy` enabled Express still sees `req.protocol === 'http'`. That value is used to build the public report URL in `getPublicReportPdf`, so shared links are generated as `http://`. The port 80 vhost proxies rather than redirecting, so plain HTTP is served; HSTS protects returning visitors but not a first contact. The certificate presented is `yigini.net.cert.combined` — confirm it actually covers the `lims` subdomain rather than relying on a wildcard that may not be there.

FIX

`RequestHeader set X-Forwarded-Proto "https"` in the 443 vhost; make the 80 vhost a 301 to HTTPS; verify the certificate's SAN list.

CLIENT-01 MEDIUM**The client's permission matrix has drifted from the server's**

```
client/src/context/AuthContext.jsx — PERMISSIONS
```

The file is labelled "Client-Side Mirror of `server/middleware/authMiddleware.js`" and is no longer a mirror. The client defines `MANAGE_EQUIPMENT`, which the server does not know; the server defines `GENERATE_REPORT` and `SHARE_REPORT`, which the client does not — and `hasPermission` returns `false` for an unknown key, so any UI gated on report generation is hidden from every user including `SUPER_ADMIN`. That is very likely a live "the button isn't there" report.

FIX

One matrix, one file, imported by both sides — or served to the client from `/api/auth/me` so drift becomes impossible.

CLIENT-02 MEDIUM**The session is rehydrated from `localStorage` and never re-validated**

```
client/src/context/AuthContext.jsx — useState initialiser
```

On load the provider reads the stored user object and trusts it. There is no call to `/api/auth/me` to confirm the token is still valid or that the role, lab and scope arrays still match the database. Editing the stored object to `"role": "SUPER_ADMIN"` unlocks the entire navigation and every gated control. Server side checks still hold on guarded routes — but

navigation and every gated control. Server-side checks still hold on guarded routes — but RBAC-06 lists the routes that are not guarded, and that is where this matters. It also means a role or lab reassignment does not take effect until the user manually logs out.

FIX

Call `/api/auth/me` on mount, use the response as the source of truth, and treat `localStorage` as a cache for first paint only.

CLIENT-03 MEDIUM**No response interceptor, so an expired token looks like a broken app**

`client/src/main.jsx:13`

There is a request interceptor that attaches the bearer token, and no response interceptor. When the 24-hour token expires mid-session every request returns 401 or 403, each page surfaces its own error state, and nothing redirects to login. To the user the application has simply stopped working — and reloading does not help, because CLIENT-02 rehydrates the same dead session from storage.

FIX

A response interceptor: on 401, clear the session and route to `/login` with a "session expired" message; on 403, show a permission notice rather than a generic failure.

OPS-03 MEDIUM**Duplicated workaround endpoints left in the request path**

`server/app.js:106` and `:630`

Two audit-log endpoints with near-identical logic, commented "*NUCLEAR OPTION: Audit Route at root level*" and "*FALLBACK: Direct Route for Audit Logs (Bypassing Router Issue)*". Their scoping rules are similar but not identical, so which one the client calls changes what a user sees. Two other pairs of this kind exist: `middleware/auth.js` is a second, weaker copy of `verifyToken` (it does not strip the password hash or parse scope arrays) that nothing currently imports but that is one careless import away from being used; and `/api/config/*` registers analysis routes twice with different permission keys.

FIX

Keep one audit route behind a proper permission key; delete the other and `middleware/auth.js`; resolve the duplicate config mounts.

AUTH-05 MEDIUM

The login rate limit is too loose to slow a password attack

server/app.js:46–51

100 requests per 15 minutes per IP, applied across all of /api/auth — so login shares its budget with /me and change-password. That permits roughly 9,600 password attempts per day from a single address against an account set that starts on a shared default password. There is no per-account lockout, no exponential backoff, and no audit entry on failed login.

FIX

A dedicated limiter on /api/auth/login — around 10 per 15 minutes keyed on IP and username together — plus an audit entry for each failure so attempts are visible.

05 What is already right

Worth stating plainly, because two of these are the patterns the rest of the remediation should copy.

- **The perimeter holds.** Fourteen endpoints probed unauthenticated — samples, users, labs, equipment, audit, admin settings, Kobo config, dashboard, reports, notifications, SIS — all returned 401. Only /api/health and the i18n bootstrap are public, and both are appropriately so.
- **Public report sharing is properly built.** 32 bytes of `crypto.randomBytes`, SHA-256 hashed at rest so the database never holds a usable token, returned to the creator exactly once, with expiry, revocation and a per-access log recording IP and user agent. This is the standard the rest of the codebase should be held to.
- **SIS API keys follow the same pattern** — hashed storage, a display prefix, expiry, activity tracking, and an `isActive` check that the user-facing login path is missing.
- **Helmet's CSP is real and specific**, enumerating the Cesium, basemap and font origins the application actually uses rather than falling back to a permissive default. HSTS, `frame-ancestors`, `object-src 'none'` and a no-referrer policy are all present and confirmed on the live response.
- **The workflow contract is genuinely shared** — server and client copies of `workflowContract.js` are byte-identical. Whatever process keeps those in step is the one to extend to the permission matrix.

- **The scope guard's design is right, even where its implementation is not.** Centralised, documented, with an explicit and logged super-admin bypass. It needs to be corrected and enforced, not replaced.

06

Implementation plan

Sequenced so that each phase makes the next one safe. Phase 0 is not optional and not parallelisable — without a committed baseline there is nothing to change against. Durations assume one developer familiar with the codebase.

PHASE 0

Establish a baseline

½ day · before anything else

Right now nobody can say what is running. Everything downstream needs that fixed first.

OPS-01

Commit the 47 changed files to a branch and push. Review the diff as you go — this is also the review the SIS module never had.

OPS-01

Verify the deployed code matches: `docker exec soilfer-lims sh -c 'find /app/server -name "*.js" -newer /app/package.json | xargs md5sum'` against the same hashes locally. Reconcile every difference before proceeding.

DATA-02

Take a verified off-host backup of the production database and confirm it opens. Do not skip this.

DATA-01

Baseline the production schema as a Prisma migration so subsequent changes are recorded.

PHASE 1

Close the nine critical findings

2–3 days · this week

Narrow, surgical patches. No refactoring — that is Phase 2. Each item is independently shippable and independently revertible.

AUTH-01

Check `isActive` in `login()`, `verifyToken()` and the WebSocket handshake. Three lines; do this first, it is the cheapest real gain here.

ISO-01

Fix the `buildScopedWhere` misuse in `spectralController`; `grep` every other call site for the same `.labId` inspection pattern.

ISO-02	Make the empty-clause path deny instead of returning <code>{}</code> , and log the misconfiguration. Expect this to surface accounts that have been relying on the fail-open — that is the finding working.
RBAC-01	Reorder <code>getUsers</code> so scope is applied last and combined with AND.
RBAC-02	Replace the <code>{ ...updates }</code> spread with a per-role field allowlist; forbid self-modification of role, lab and scope arrays.
RBAC-03	Validate <code>role</code> against a constant on create and update; define which roles each actor role may grant.
SIS-01/02	Intersect caller filters with the key's scope instead of letting them override it; scope <code>getSpectra</code> and <code>getStats</code> . Audit <code>ApiKey</code> rows for keys issued with a null (unrestricted) scope while you are in there.
ISO-03	Point <code>/api/dashboard/live</code> at the shared guard so its numbers agree with the sample list.

PHASE 2 One authorization layer

1 week

Phase 1 patches nine holes. This phase removes the conditions that produced them — so that the next controller written is safe by construction rather than by review.

RBAC-07	Create <code>shared/roles.js</code> : the canonical role list, each role's permissions, and display names. Import it in the server middleware, the client context, <code>seed.js</code> and the lab-provisioning route. Migrate or remove <code>ADMIN</code> , <code>COUNTRY_ADMIN</code> , <code>INTAKE_OFFICER</code> ; decide what <code>AUDIT_USER</code> and <code>EXTERNAL_VIEWER</code> may actually do and give them permissions, or stop provisioning them.
CLIENT-01	Serve the effective permission set from <code>/api/auth/me</code> so the client cannot drift from the server again.
RBAC-05/06	Require every route to declare a permission key and a scope policy. A route registered without both should fail at startup, not silently pass traffic — that assertion is what makes the guarantee stick.
RBAC-04	Delete <code>checkScope</code> and <code>requireLabScope</code> , or wire them and invert their fail-open conditions. Delete <code>middleware/auth.js</code> .
OPS-03	Remove the duplicate audit endpoint and the duplicate config mounts.
AUTH-	Add <code>tokenVersion</code> , the <code>mustChangePassword</code> gate, and the impersonation act claim

02/03/04

with audit entries and a 30-minute cap.

QUAL-01

Write tests/setup.js, get the eleven existing suites green, and add one regression test per Phase 1 finding. Ideally this lands alongside Phase 1 rather than after it.

PHASE 3 Repair the connections

1 week

The features that are documented but do not run. This is the phase users will actually notice.

INT-01

Build the Kobo scheduler: per-lab interval, per-lab last-run and last-error surfaced in the admin UI so a dead connection is visible instead of silent.

INT-02

Give the Kobo client a timeout, pagination, backoff retry, and a persisted high-water submission id.

INT-03

Install puppeteer-core and Alpine chromium in the image; make the missing-browser branch a 503 rather than a 200 carrying JSON.

WS-01/02

Add broadcastToLab and replace every broadcastToAll call; move the WS token out of the query string; scope the online-user list. Expect a visible responsiveness improvement from the reduced refetch churn.

CLIENT-02/03

Validate the session against /api/auth/me on mount; add the 401/403 response interceptor.

NET-01

Set X-Forwarded-Proto, redirect port 80 to 443, verify the certificate covers the subdomain.

PHASE 4 Make deployment repeatable

1 week

So that OPS-01 cannot recur, and so the earlier phases stay fixed.

OPS-02

CI on push: install, lint, run the now-working test suite, build the client, build a tagged image. A red build blocks the deploy.

OPS-02

Deploy by pulling that tagged image. Keep the previous tag so rollback is one command. Retire the scp and docker cp procedure and the deploy runbook that documents it.

DATA-01

Run prisma migrate deploy as an explicit deployment step; the volume-overlay and dual-upload workaround disappears once the schema ships in the image.

DATA-02	Move backups to a cron outside the app process using <code>VACUUM INTO</code> , verify each one opens, and copy off-host. Then evaluate PostgreSQL against measured write contention rather than in the abstract.
AUTH-05	Tighten the login limiter and add audit entries for failed logins.
–	Rotate <code>JWT_SECRET</code> and every SIS API key once Phase 1–3 are deployed, on the assumption that the fail-open period may have leaked material.

07 Scope and limits of this audit

What was verified, and how

- **Source review** — complete, against the working tree on the workstation: 31 controllers, 26 route modules, all auth and scope middleware, the WebSocket server, the Prisma schema, the client auth context and route table, the Docker and Apache configuration, and the deployment runbook. Every finding cites the file it came from.
- **Live perimeter** — verified read-only against `https://lims.yigini.net`: fourteen unauthenticated endpoint probes, response headers, TLS and CSP, deployed bundle timestamp. Nothing was written, and no account was used.
- **Not verified: authenticated runtime behaviour.** The escalation paths in RBAC-01, RBAC-02 and SIS-01 are established by reading the code, not by exercising them against production — that would have required credentials and would have written to the audit log. They should be confirmed against a staging copy before and after the fix.
- **Not verified: file-level parity between this working tree and the container.** The deployment procedure copies individual files, so the container may hold a different mix again. The Phase 0 hash comparison settles it; until then, treat every finding as "present in the code that appears to be deployed" rather than as confirmed on the running instance.
- **Not covered:** dependency CVE scanning, the analytical correctness of the QC and validation logic, i18n completeness, and load or performance testing. The DATA-02 contention concern is reasoned from the architecture, not measured.